

High-Assurance Execution Substrates for Autonomous Agents: Architecting the olmxplusplus.rs Runtime Environment

Introduction to the Agentic Computing Paradigm and Security Crisis

The rapid evolution of artificial intelligence has precipitated a fundamental paradigm shift in software engineering and systems architecture. The industry is actively transitioning from static, single-pass autoregressive text generation toward autonomous, multi-agent systems.¹ These advanced intelligent entities possess the capacity for complex environmental perception, strategic self-planning, and autonomous action execution over extended temporal horizons. However, integrating such autonomous AI agents into enterprise production environments has catalyzed a phenomenon widely described as the world's largest shadow information technology problem.¹ The contemporary practice of end-users rapidly prototyping applications using AI—often termed "vibe coding"—without rigorous architectural oversight results in the uncontrolled proliferation of unverified code, the inadvertent exposure of sensitive cryptographic credentials, and the deployment of software completely lacking fundamental security audits.¹

Translating agent-generated prototypes into robust, production-ready software introduces severe operational friction. This friction originates from a core architectural mismatch: autonomous agents are traditionally forced to operate within programming languages and execution runtimes designed exclusively for human cognitive models.¹ Traditional programming paradigms prioritize human-readable syntax, dynamic typing for developer velocity, and complex, text-based debugging mechanisms tailored for human comprehension.¹ In stark contrast, Large Language Models (LLMs) and autonomous reasoning engines struggle with implicit state, require highly structured and deterministic feedback loops, and introduce unprecedented system-level security vulnerabilities when granted execution privileges in legacy, human-oriented environments.¹

More critically, deploying autonomous coding agents within standard environments exposes underlying infrastructure to catastrophic security vulnerabilities. In an agentic framework equipped with tool-calling capabilities and a native compiler or interpreter, a successful prompt injection translates directly into arbitrary Remote Code Execution (RCE).¹ Academic evaluations of state-of-the-art LLMs functioning as reasoning engines reveal that over 94 percent of these models succumb to Direct Prompt Injection designed specifically to achieve complete

computer takeover, while over 83 percent fail against Retrieval-Augmented Generation (RAG) backdoors that autonomously install malware.¹ Standard containerization strategies, such as Docker, rely on Software Fault Isolation (SFI) operating atop a shared host operating system kernel, utilizing Linux namespaces and cgroups to partition resources.¹ Consequently, an adversarial agent that successfully generates a kernel exploit can shatter the container boundary and compromise the underlying infrastructure, as the workload is functionally only one system call away from complete host compromise.¹

To mitigate these systemic risks and unlock the true potential of agentic computing at the edge, this research report and Product Requirements Document (PRD) outlines a highly deterministic, resource-constrained execution framework. Specifically, this architecture defines the execution perimeter for `olmxplusplus.rs`—a highly optimized, Rust-native port of the oMLX inference engine.¹ Engineered for local-only, audit-ready daemon deployments on Raspberry Pi 5-class Linux edge devices, `olmxplusplus.rs` represents a secure foundation for edge AI.¹ By synthesizing WebAssembly (WASM) sandboxing, deterministic filesystem and network virtualization, eBPF-based tool-call resource constraints, and formal capability-based security protocols, this document provides the definitive specification for safely executing untrusted AI logic and managing local-agent interactions on constrained hardware topologies.

The Inference Engine: Architecting `olmxplusplus.rs` for the Edge

To understand the specific execution requirements of the agent runtime, it is first necessary to define the primary inference target. The `olmxplusplus.rs` initiative is a direct architectural evolution of the oMLX framework. Originally built for Apple Silicon, oMLX leveraged the MLX framework to treat CPU and GPU memory as a single unified pool, delivering significantly faster and more memory-efficient inference than traditional Metal-based alternatives like Ollama.³ A hallmark feature of the original oMLX architecture is its advanced Paged SSD KV caching system.⁴ By persistently storing Key-Value (KV) cache blocks across a hot in-memory tier and a cold SSD tier, the engine allows past context to remain cached and reusable across requests, even when the LLM context changes mid-conversation.⁶ This block-based approach, utilizing prefix sharing and Copy-on-Write mechanisms, effectively drops the agent Time To First Token (TTFT) from 30–90 seconds down to under 5 seconds, making local LLMs highly practical for continuous, long-running coding tasks.⁴

The `olmxplusplus.rs` port represents a server-parity rewrite slice explicitly designed for high-assurance, local-only, audit-ready daemon operations targeting Raspberry Pi 5-class Linux devices.¹ Operating within the constraints of edge hardware, the Rust port replaces the Python-centric FastAPI and PyObjC layers of its predecessor with a highly concurrent, memory-safe Rust backend.¹

The core architecture of the `olmxplusplus.rs` daemon is governed by the `InferenceRuntime` trait.¹ This trait abstracts the underlying hardware acceleration, allowing the engine to function as a highly flexible fixture runtime.¹ The implementation retains the critical versioned SSD cache

metadata primitives that made oMLX highly performant for agentic reasoning loops, ensuring that complex context trees can be suspended and resumed locally without massive re-computation penalties.¹ The API surface is rigorously tested for JSON shape compatibility, providing native OpenAI-shaped endpoints (including /health, /v1/models, /v1/chat/completions, and /v1/completions), allowing seamless drop-in replacement for orchestration frameworks relying on cloud APIs.¹

Furthermore, the olmxplusplus.rs ecosystem is explicitly engineered for auditability. The runtime includes a deeply integrated threat model, meticulous runtime decision logic documentation, strict behavior baselines, and an audit checklist designed to satisfy enterprise compliance requirements.¹ For hardware acceleration on constrained devices, the architecture tracks optional support for the Coral Edge TPU, functioning as a "small-model coprocessor lane" designed to offload specific reasoning or embedding tasks, though it fundamentally bypasses direct execution of GGUF or ONNX artifacts to maintain strict memory formatting.¹

The WebAssembly Execution Substrate and the Cranelift JIT

With the olmxplusplus.rs inference engine providing the underlying cognitive reasoning, the generated code and tool calls must be executed within an impenetrable boundary. Traditional Python-based agent frameworks typically execute tools by spawning subprocesses or calling functions directly within the same host process.⁸ This completely negates sandbox boundaries; a tool with a logic bug or a malicious payload injected via the prompt can trivially access the local filesystem, initiate unauthorized network requests, or scrape memory for API keys belonging to other agent sessions.⁸

WebAssembly Linear Memory Isolation

To safely execute agent-generated logic, the architecture mandates the use of WebAssembly (WASM). WebAssembly provides an architecture-neutral binary instruction format that operates on a shared-nothing "Linear Memory" model.¹ When a WASM module representing the compiled logic of an agent is instantiated, it is allocated a single, contiguous block of raw bytes.¹ The underlying execution engine enforces strict, mathematically verifiable bounds checking, cryptographically ensuring that every memory load or store instruction remains strictly within the `0` to `max_memory` range defined at instantiation.¹ This architectural design physically prevents the executing module from addressing the memory of the host process, the olmxplusplus.rs daemon, or any adjacent agent sandboxes operating on the same node.¹

For edge deployments targeting Raspberry Pi 5 systems, relying on traditional hardware-level virtualization, such as Firecracker MicroVMs, introduces prohibitive resource overhead and initialization latency.¹ While MicroVMs offer robust perimeter defense by provisioning a dedicated, lightweight Linux kernel per sandbox, their startup times—even optimized to milliseconds—and heavy orchestration complexity involving nomads and control planes

degrade the rapid iterative feedback loops required by autonomous agents.¹ WASM sandboxing circumvents this by offering micro-VM levels of isolation with container-level execution velocity, achieving cold start times of just 50 microseconds over native C implementations.¹

Compiler Security: The Mandate for Cranelift

To execute dynamic agent logic within this sandbox rapidly, the runtime must integrate a Just-In-Time (JIT) compiler. Traditional Ahead-of-Time (AOT) compilation introduces latency barriers that disrupt the agent's real-time reasoning loop.¹ However, utilizing industry-standard JIT backends like LLVM introduces severe structural vulnerabilities when applied to adversarial AI outputs.¹ LLVM, comprising over 20 million lines of complex C++ code, focuses on aggressive AOT optimization and relies extensively on "Undefined Behavior" (UB) within its Intermediate Representation (IR) to strip away logic it deems mathematically impossible.¹ Feeding adversarial, agent-generated IR into LLVM exposes the host to sophisticated compiler-crashing logic bugs, denial-of-service states, and subtle miscompilations.¹

Therefore, the olmxplusplus.rs execution substrate is explicitly designed to utilize the Cranelift framework.¹ Developed by the Bytecode Alliance, Cranelift is a highly secure compiler backend written entirely in safe Rust, fundamentally eliminating the buffer overflow vulnerabilities inherent to C++ compilers.¹ Cranelift is explicitly hardened against malicious inputs, mimicking the defensive posture of modern browser JIT engines.¹ It utilizes input-length-bounded recursion and strictly avoids algorithms with quadratic or higher time complexities, neutralizing any attempt by the LLM to generate code designed to stall the compiler.¹

Most critically, Cranelift's IR is meticulously designed without undefined behavior, ensuring perfect determinism; the code executes exactly as specified, facilitating deterministic auditing and repair loops.¹ Cranelift prioritizes compilation speed, performing code-generation operations an order of magnitude faster than equivalent LLVM-based systems, achieving sub-200-millisecond compilation latencies.¹ This velocity is essential for the ephemeral, IO-bound tool executions generated by olmxplusplus.rs agents.

Architectural Domain	Legacy Approach (LLVM)	High-Assurance Target (Cranelift)
Implementation Language	C++ (Memory-unsafe)	Safe Rust
Codebase Footprint	Massive (~ 2 Million LOC)	Lightweight (~ 200,000 LOC)
Compilation Velocity	High latency (AOT focus)	Microsecond latency (JIT focus)
Undefined Behavior (UB)	Permitted and relied upon for optimization	Strictly eradicated from IR
Malicious Input Hardening	Low (Assumes trusted source code)	High (Linear time algorithms, bounded recursion)

Deterministic Filesystem and Network Virtualization

Confining the agent's logic to a linear memory sandbox is only the first step; the agent must eventually interact with the outside world to execute useful tasks. However, exposing the host operating system's raw POSIX interfaces creates an uncontrollable attack surface. The architecture requires a deterministic firewall to control exactly how the agent interacts with storage and networking.

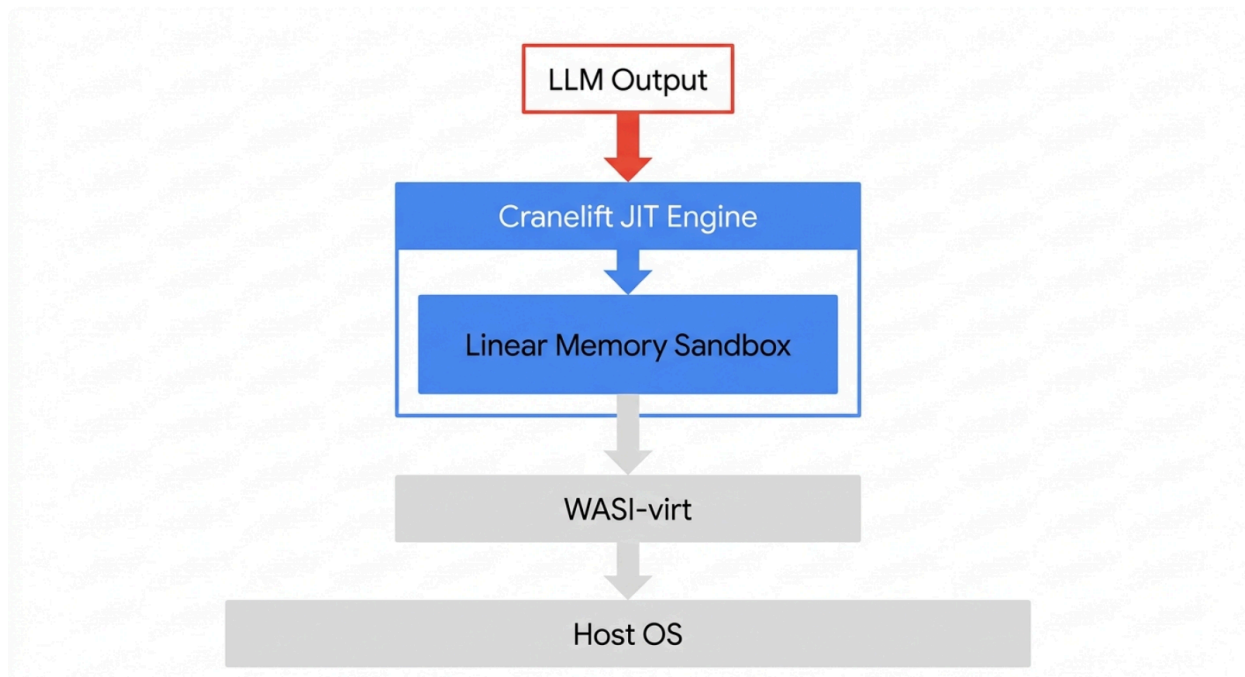
Eradicating Filesystem Non-Determinism via WASI-virt

Standard operating system filesystems are inherently non-deterministic and highly idiosyncratic. Different operating systems impose disparate rules regarding valid node names, maximum path lengths, and character encoding.¹¹ For instance, legacy FAT16 limits paths to 8.3 naming conventions, while Windows environments retain highly specific restrictions regarding magical files (e.g., COM1, LPT7) and file streams (e.g., some_file.txt:stream1).¹¹ If an agent's code interacts directly with the host filesystem, its execution path becomes entirely dependent on the host's underlying rules, breaking cross-platform determinism and allowing the agent to dynamically probe filesystem vulnerabilities.¹¹

To enforce absolute determinism and security, the runtime implements WebAssembly System Interface Virtualization (WASI-virt).¹² WASI-virt establishes a sophisticated, two-tier filesystem access model that operates as an impenetrable firewall.¹¹ Rather than utilizing mechanisms to expose raw host paths to the WASM module, specific file paths are mounted and fully virtualized into the component as a post-compile operation.¹²

Within this architecture, the inner program generated by the agent still imports a standard WASI filesystem interface, believing it is interacting with a standard drive. However, the implementation of that filesystem is entirely provided by a secondary, highly trusted Rust component, not the actual host environment.¹² This allows the host to map complex, non-deterministic host paths to simplified, deterministic virtual node identifiers (e.g., mapping a host directory to a virtual node 0xDEADBEEF internally).¹¹ The final composed WebAssembly component is structurally banned from invoking raw host filesystem APIs, ensuring that even if the agent attempts directory traversal attacks, it remains trapped within a synthetic, read-only virtual structure.¹¹

Secure Compilation and Virtualization Pipeline



Untrusted agent logic is compiled via the UB-free Craneflift JIT into a WebAssembly linear memory sandbox. All filesystem and network interactions are intercepted by WASI-virt, providing a fully deterministic, virtualized OS boundary.

Pre-Action Authorization and the MCP Gateway

Filesystem virtualization solves local access, but autonomous agents require network access to execute APIs, delegate sub-tasks, and retrieve data. Traditional network firewalls rely on IP and port filtering, which are insufficient for agent security because agents frequently use legitimate messaging platform servers or standard HTTPS ports to execute malicious actions.¹³

Furthermore, providing an agent with a static API key creates a massive vulnerability; exposed secrets within agent memory are easily harvested via prompt injection.¹⁴

To resolve this, the `olmxplusplus.rs` runtime utilizes the Model Context Protocol (MCP) as its exclusive network gateway.¹ Agents are structurally prohibited from accessing the raw network stack; they cannot invoke `curl` or open arbitrary TCP sockets. Instead, they must invoke pre-registered MCP tools via a structured JSON-RPC 2.0 interface.¹ The runtime intercepts the invocation, strips it of memory pointers, serializes the arguments, and routes the payload to the external service.¹

While MCP structures the interaction, it does not evaluate the *intent* of the action. Current safety architectures rely on probabilistic model alignment or post-hoc evaluation, neither of which provide deterministic enforcement before a network action occurs.¹⁵ To close this

vulnerability, the architecture mandates the integration of the Open Agent Passport (OAP) specification.¹⁵

OAP provides a synchronous, deterministic pre-action authorization firewall.¹⁵ Operating as a blocking hook (before_tool_call), OAP intercepts every MCP tool invocation before execution and evaluates the parameters against a declarative, versioned specification known as a Policy Pack.¹⁷ This evaluation is programmatic and entirely deterministic; it uses no LLM sampling or temperature, ensuring that prompt injection cannot bypass the rules.¹⁷ Furthermore, OAP produces a cryptographically signed audit record for every decision, binding the agent's identity to its authorized capability scope.¹⁵

In empirical adversarial testbeds evaluating 4,437 authorization decisions, social engineering attacks against the base LLM succeeded 74.6 percent of the time under permissive policies.¹⁵ However, by implementing OAP's deterministic firewall, a comparable population of attackers achieved a 0 percent success rate across 879 top-tier breach attempts, with the enforcement adding a median latency of merely 53 milliseconds.¹⁵

Defense Mechanism	Enforcement Stage	Evaluation Method	Adversarial Success Rate (Empirical)
Model Alignment / Permissive	Reasoning (Probabilistic)	LLM Internal Weights	74.6%
Open Agent Passport (OAP)	Pre-Action (Deterministic)	Declarative Policy Pack	0.0%

Tool-Call Granularity Resource Firewalls via eBPF

In addition to network and filesystem constraints, the olmxplusplus.rs daemon must enforce strict computational resource limits. Autonomous agents frequently encounter reasoning loops that generate massive volumes of intermediate data, leading to severe resource monopolization. Traditional container-level resource limits (like Docker's static RAM ceilings) fail catastrophically when applied to agentic workloads.¹⁸

The Tripartite Resource Mismatch

A systematic characterization of OS-level resource dynamics analyzing 144 software engineering tasks from the SWE-rebench benchmark reveals three critical mismatches between AI agent behavior and legacy resource controls¹⁸:

- Granularity Mismatch:** Agent workloads exhibit a highly bifurcated memory profile. There is a stable baseline for the framework (approximately 185 MB), which is punctuated by extreme, tool-call-driven memory bursts.²¹ Applying a flat, container-level policy either starves the tool call or provisions massive amounts of unused baseline memory.¹⁸
- Responsiveness Mismatch:** Agent resource bursts are incredibly brief, often lasting only 1 to 2 seconds, but feature peak-to-average consumption ratios up to $15.4\times$ the baseline.¹⁸ Traditional user-space resource watchers cannot react to these sub-second

unpredictable bursts fast enough, leading to system degradation.¹⁸

3. **Adaptability Mismatch:** Unlike predictable microservices, agent execution is stateful and highly non-deterministic.¹⁸ History-based prediction fails because the agent's next tool call is entirely dependent on its stochastic reasoning process.¹⁸ Furthermore, memory is identified as the primary concurrency bottleneck, while average CPU utilization remains low (7.6–13.2%).²²

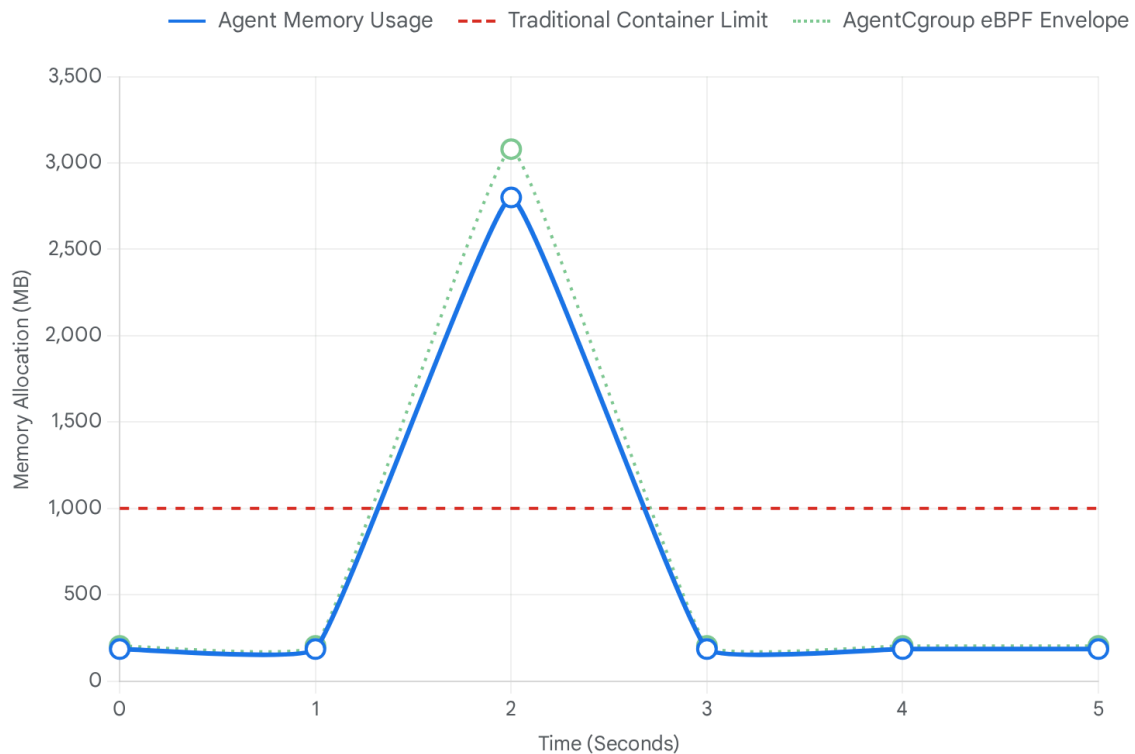
AgentCgroup: In-Kernel Enforcement

To implement a deterministic resource firewall, the architecture integrates the design principles of AgentCgroup, an intent-driven resource controller utilizing the Extended Berkeley Packet Filter (eBPF).¹⁸ AgentCgroup organizes resources using a hierarchical cgroup v2 structure, mapping the primary agent workload to a parent node while dynamically generating child nodes for individual tool calls.²⁴ This provides granular control without destroying the overarching agent context.²⁴

Crucially, the control logic executes directly within the Linux kernel at the cgroup enforcement points via eBPF, eliminating user-kernel context switching and achieving microsecond responsiveness.²⁴ For CPU scheduling, the sched_ext eBPF module prioritizes latency-sensitive tool calls and implements automatic fail-safe reversion.²⁴ For memory, which is the critical bottleneck, memcg_bpf_ops hooks trace process creation and memory allocation in real time.²⁴

When a tool call breaches its designated soft limit (memory.high), the BPF program does not immediately terminate the process. Instead, it applies graduated throttling delays.²⁴ If memory pressure escalates dangerously close to the hard limit (memory.max), the system invokes the cgroup.freeze primitive.²⁴ This freezes the specific tool's subtree atomically, preserving the agent's valuable reasoning context while allowing the orchestrator to resolve the conflict.²⁴ Evaluations of this architecture demonstrate a 29 percent reduction in high-priority P95 allocation latency under moderate memory conditions, with only a 0.3 percent increase in P50 enforcement overhead.²²

Agent Memory Bursts vs. Resource Control Limits



AI agent tool calls exhibit massive, sub-second memory spikes (up to 15.4x baseline). AgentCgroup utilizes eBPF and cgroup v2 to dynamically wrap these bursts, preventing the responsiveness mismatch inherent in static container limits.

Data sources: [arXiv \(seL4 Microkit\)](#), [Medium](#), [arXiv \(AgentCgroup Architecture\)](#), [The Moonlight](#)

High-Assurance RTOS Foundations: seL4 and LionsOS

While eBPF provides robust resource control on standard Linux kernels, deploying `olmxplusplus.rs` in mission-critical environments requires an operating system architecture that is mathematically proven to be secure. Standard monolithic kernels present an attack surface that is structurally too broad.¹ Therefore, the architecture supports operation atop the seL4 microkernel and the LionsOS framework.

seL4 Mixed Criticality Scheduling (MCS)

The seL4 microkernel is universally recognized as the world's most highly assured operating system, featuring formal mathematical verification of its functional correctness, integrity, and

confidentiality.¹ The `olmxplusplus.rs` runtime utilizes the `seL4 Microkit`, a framework designed to build systems with a static architecture where system resources are rigorously assigned upfront during initialization.²⁷ The `Microkit` isolates agent components into distinct Protection Domains (PDs), which communicate exclusively through explicit, capability-based channels.²⁷ To replace the resource constraints provided by `eBPF` in a Linux environment, the `seL4` implementation utilizes Mixed Criticality Systems (MCS) scheduling contexts.²⁶ MCS introduces a deterministic abstraction for managing CPU execution time based on the sporadic server algorithm.³⁰ The fundamental parameters of a Scheduling Context (SC) are its period T and its total budget C , where $C \leq T$.³⁰

When a protection domain requires execution time, a `schedControl` capability is used to invoke `seL4_SchedControl_Configure`, legally binding the SC parameters to the agent's thread.³⁰

- **Full Scheduling Contexts:** Configured where $C = T$, granting the agent access to 100 percent of the CPU time during that period, mimicking round-robin execution.³¹
- **Partial Scheduling Contexts:** Configured where $C < T$, establishing a strict upper bound on execution time, guaranteeing that a runaway LLM loop cannot starve critical system operations.³¹

If an agent exceeds its allocated budget, it triggers a Timeout Fault.³³ The `seL4 Monitor PD` intercepts this fault, delivering a precise message containing the scheduling context data word and the time consumed.³³ The system orchestrator can then deterministically reboot, pause, or adjust the agent's budget, ensuring total system stability without relying on unpredictable user-space heuristics.³³

LionsOS Metaprogrammed Network Firewalls

Executing WebAssembly toolchains atop `seL4` requires a secure networking stack that standard microkernels natively lack. The architecture achieves this by integrating `LionsOS`, an operating system personality built directly atop the `seL4 Microkit`.¹ `LionsOS` eschews monolithic complexity, ensuring components perform single, highly defined functions and communicate via lock-free queues.³⁵

To enforce network determinism at the lowest level, `LionsOS` implements a highly modular, metaprogrammed firewall.³⁶ The firewall's routing tables, protocol filter components (UDP, TCP, ICMP), and data structure capacities are statically configured via a Python `sdfgen` script (`meta.py` and `constants.py`) during the build process.³⁶

The architecture physically isolates Address Resolution Protocol (ARP) operations. It deploys an ARP Requester to handle outbound IP mappings and a distinct, simplistic ARP Responder that blindly answers matching requests based on its statically defined MAC address, completely isolating the agent from network discovery attacks.³⁷ All outbound traffic generated by the agent's WASM engine is forced through an Rx Virtualiser, which multiplexes the packets through the statically defined protocol filters before they can reach the hardware driver.³⁷ Because these firewall rules are constructed via metaprogramming and operate in isolated

PDs, the agent cannot dynamically alter the routing table, guaranteeing deterministic network egress.³⁶

Deterministic Control of Local-Agent Interactions

As the `olmxplusplus.rs` daemon orchestrates multiple specialized models (LLMs, VLMs, and embedding models) concurrently⁷, local-agent interactions become a critical vulnerability vector. This interconnectedness catalyzes "emergent swarm intelligence," but it simultaneously introduces severe lateral movement risks.³⁸ If an external or compromised agent interfaces with the local assistant, persuasive framing, urgency tactics, or embedded prompt injections can propagate horizontally, creating emergent threats that no single agent could generate in isolation.³⁸

To securely govern these interactions, the architecture implements a **Dual-Firewall Inter-Agent Communication Protocol**.⁴⁰ This framework mathematically projects the communication onto the specific task context, guaranteeing that only safe, strictly necessary information crosses the trust boundary.⁴⁰

The Language Converter Firewall (Inbound)

When an external agent transmits a free-form natural language message to the local assistant, it is intercepted by the Language Converter Firewall.⁴¹ This inbound filter removes the asymmetric challenge of trying to detect endless variations of prompt injections by structurally destroying the attack channel.⁴⁰ It operates via three deterministic stages:

1. **LLM-Based Protocol Conversion:** An isolated model converts the external agent's unstructured text into a closed, domain-specific structured JSON protocol.⁴³
2. **Deterministic Verification:** A programmatic verifier rigorously validates every field. It enforces a strict closed vocabulary (dropping keys like `system_override`), checks enumerated fields against allowlists, and guarantees type safety.⁴³
3. **String Anonymization:** To prevent attackers from hiding injections within legitimate data fields, free-form strings (such as hotel names or company identifiers) are replaced with anonymous, deterministic identifiers (e.g., `hotel_1`) before reaching the assistant.⁴³

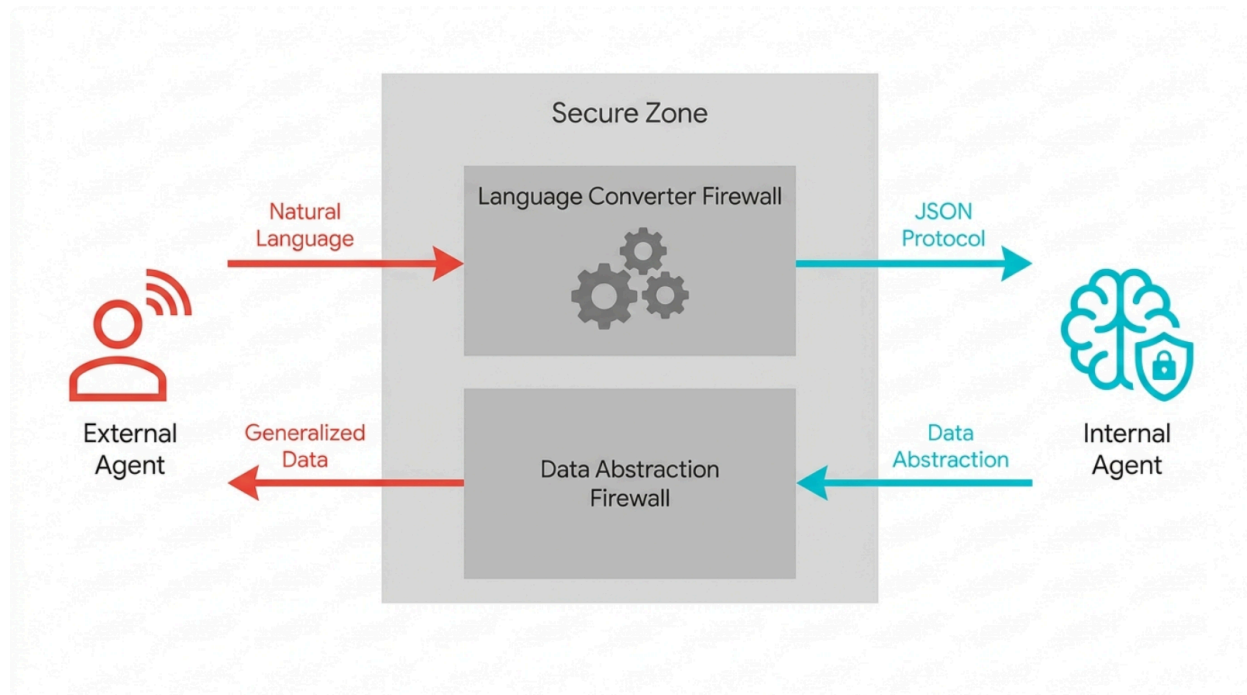
The Data Abstraction Firewall (Outbound)

Conversely, when the local agent retrieves data to transmit back to an external entity, LLMs inherently tend to overshare context, exposing sensitive local information.⁴³ The Data Abstraction Firewall intercepts these outgoing payloads and transforms the raw data into task-appropriate abstractions rather than utilizing binary block/allow lists.⁴¹ For example, it will project a precise geographic location into a broad regional category, or convert exact financial figures into ranges, thereby operationalizing contextual integrity.⁴¹

Empirical evaluation on the comprehensive ConVerse benchmark highlights the necessity of this architecture. Across 864 distinct attacks, the implementation of the dual-firewall protocol reduced privacy attack success rates drastically (from 84 percent down to 10 percent), while nearly eliminating security attack success (reducing it from 60 percent to 3 percent), all while

maintaining high task completion quality.⁴⁰

Deterministic Inter-Agent Communication Firewalls



The Language Converter Firewall neutralizes inbound prompt injections by converting natural language into a deterministically verified JSON protocol. Concurrently, the Data Abstraction Firewall prevents outbound oversharing by generalizing contextual data.

Architectural Synthesis and Conclusion

The integration of autonomous, agentic reasoning engines into mission-critical edge infrastructure requires a severe departure from legacy execution environments. Relying on traditional operating system constructs, human-centric compilers, and basic containerization strategies exposes the underlying hardware to unacceptable risks, primarily through prompt-injection-driven remote code execution and unauthorized lateral movement. The `olmxplusplus.rs` architecture establishes a comprehensive, high-assurance execution substrate tailored precisely for the unique operational constraints of AI agents on hardware like the Raspberry Pi 5. By confining all untrusted reasoning logic within a mathematically rigorous WebAssembly linear memory sandbox, and compiling that logic via the UB-free Cranelift JIT compiler, the runtime guarantees memory safety and achieves microsecond deployment latencies. Host interactions are completely decoupled from legacy POSIX vulnerabilities; the filesystem is virtualized into a deterministic sandbox via WASI-virt, while network execution is structurally bounded by the Model Context Protocol (MCP) and cryptographically authorized

by the Open Agent Passport (OAP) specification.

Furthermore, the architecture resolves the severe resource consumption volatility inherent to LLM tool calls. Whether deploying AgentCgroup eBPF hooks on Linux to orchestrate dynamic, sub-second cgroup boundaries, or utilizing the formally verified seL4 microkernel and LionsOS to enforce Mixed Criticality Scheduling and metaprogrammed network rules, the daemon ensures deterministic resource allocation. Finally, by securing the emergent swarm intelligence of the network via a dual-firewall protocol that structurally anonymizes and verifies inter-agent communication, the olmxplusplus.rs framework ensures that multi-agent edge intelligence operates within an impenetrable, highly auditable perimeter.

Works cited

1. aegis-os-1.0.pdf
2. WebAssembly could solve AI agents' most dangerous security gap - The New Stack, accessed May 17, 2026, <https://thenewstack.io/webassembly-sandboxing-ai-agents/>
3. Running oMLX on Apple Silicon — Installation, Models, and Shell Management - Medium, accessed May 17, 2026, <https://medium.com/@sami.bister/running-omlx-on-apple-silicon-installation-models-and-shell-management-aa50f06aff2d>
4. oMLX — LLM inference, optimized for your Mac, accessed May 17, 2026, <https://omlx.ai/>
5. oMLX - open-source MLX inference server with paged SSD caching for Apple Silicon : r/LocalLLaMA - Reddit, accessed May 17, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1r3qwyi/omlx_opensource_mlx_inference_server_with_paged/
6. GitHub - jundot/omlx: LLM inference server with continuous batching & SSD caching for Apple Silicon — managed from the macOS menu bar, accessed May 17, 2026, <https://github.com/jundot/omlx>
7. oMLX: Local LLM inference server for Apple Silicon with continuous batching ... - Jimmy Song, accessed May 17, 2026, <https://jimmysong.io/ai/omlx/>
8. From OpenClaw to Agentor: Building Secure AI Agents in Rust | Xcapit, accessed May 17, 2026, <https://www.xcapit.com/en/blog/from-openclaw-to-agentor-building-secure-ai-agents-in-rust>
9. Building Complex Agentic Systems with WebAssembly - Tamal Dutta Chowdhury, accessed May 17, 2026, <https://tamal.tech/building-complex-agentic-systems-with-webassembly/>
10. Sandboxing Agentic AI Workflows with WebAssembly | NVIDIA Technical Blog, accessed May 17, 2026, <https://developer.nvidia.com/blog/sandboxing-agentic-ai-workflows-with-webassembly/>
11. Roadmap to determinism in WASI · Issue #190 · WebAssembly/WASI - GitHub, accessed May 17, 2026, <https://github.com/WebAssembly/WASI/issues/190>
12. bytocodealliance/WASI-Virt: Virtual implementations of WASI APIs - GitHub,

- accessed May 17, 2026, <https://github.com/bytecodealliance/wasi-virt>
13. Toward Securing AI Agents Like Operating Systems - arXiv, accessed May 17, 2026, <https://arxiv.org/html/2605.14932v1>
 14. How Autonomous AI Agents Like Openclaw are Reshaping Enterprise Identity Security, accessed May 17, 2026, <https://www.paloaltonetworks.com/blog/identity-security/how-autonomous-ai-agents-like-openclaw-are-reshaping-enterprise-identity-security/>
 15. Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents - arXiv, accessed May 17, 2026, <https://arxiv.org/pdf/2603.20953>
 16. [2603.20953] Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents - arXiv, accessed May 17, 2026, <https://arxiv.org/abs/2603.20953>
 17. Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents, accessed May 17, 2026, <https://arxiv.org/html/2603.20953v1>
 18. AgentCgroup: Understanding and Controlling OS Resources of AI Agents - arXiv, accessed May 17, 2026, <https://arxiv.org/pdf/2602.09345>
 19. AgentCgroup: Understanding and Controlling OS Resources of AI Agents - GitHub, accessed May 17, 2026, <https://github.com/eunomia-bpf/agentcgroup>
 20. [2602.09345] AgentCgroup: Understanding and Controlling OS Resources of AI Agents, accessed May 17, 2026, <https://arxiv.org/abs/2602.09345>
 21. AgentCgroup: Understanding and Controlling OS Resources of AI Agents - arXiv, accessed May 17, 2026, <https://arxiv.org/html/2602.09345v2>
 22. [Literature Review] AgentCgroup: Understanding and Controlling OS Resources of AI Agents - Moonlight, accessed May 17, 2026, <https://www.themoonlight.io/en/review/agentcgroup-understanding-and-controlling-os-resources-of-ai-agents>
 23. [PDF] AgentCgroup: Understanding and Controlling OS Resources of AI Agents, accessed May 17, 2026, <https://www.semanticscholar.org/paper/AgentCgroup%3A-Understanding-and-Controlling-OS-of-AI-Zheng-Fan/d9c4a1cdcdb7981b986f888b78e93d05886789fa>
 24. AgentCgroup: What Happens When AI Coding Agents Meet OS Resources? | by yunwei37, accessed May 17, 2026, <https://medium.com/@yunwei356/agentcgroup-what-happens-when-ai-coding-agents-meet-os-resources-46ea027ef62d>
 25. seL4 - Wikipedia, accessed May 17, 2026, <https://en.wikipedia.org/wiki/SeL4>
 26. seL4 Reference Manual Version 15.0.0, accessed May 17, 2026, <https://sel4.systems/Info/Docs/seL4-manual-latest.pdf>
 27. microkit/docs/manual.md at main - GitHub, accessed May 17, 2026, <https://github.com/sel4/microkit/blob/main/docs/manual.md>
 28. Microkit User Manual (2.2.0) | seL4 docs, accessed May 17, 2026, <https://docs.sel4.systems/projects/microkit/manual/latest/>
 29. The seL4 Microkernel – An Introduction, accessed May 17, 2026, <https://sel4.systems/About/seL4-whitepaper.pdf>
 30. Strengthening scheduling guarantees of seL4 MCS with IPC budget limits Mitchell Johnston - Trustworthy Systems, accessed May 17, 2026,

- https://trustworthy.systems/publications/theses_public/22/Johnston%3Abe.pdf
31. MCS - seL4 docs, accessed May 17, 2026, <https://docs.sel4.systems/Tutorials/mcs.html>
 32. seL4 MCS Reference Manual Version 10.1.1-MCS, accessed May 17, 2026, <https://sel4.systems/Info/Docs/seL4-manual-10.1.1-mcs.pdf>
 33. seL4 Reference Manual Version 12.0.0-dev, accessed May 17, 2026, <https://www.cse.unsw.edu.au/~cs9242/21/project/seL4-manual.pdf>
 34. Fast, Secure, Adaptable: LionsOS Design, Implementation and Performance - arXiv, accessed May 17, 2026, <https://arxiv.org/html/2501.06234v1>
 35. Introduction | LionsOS 0.3.0, accessed May 17, 2026, <https://lionsos.org/>
 36. Building | LionsOS 0.3.0, accessed May 17, 2026, <https://lionsos.org/docs/examples/firewall/building/>
 37. Firewall | LionsOS 0.3.0, accessed May 17, 2026, <https://lionsos.org/docs/examples/firewall/>
 38. From Thinker to Society: Security in Hierarchical Autonomy Evolution of AI Agents - arXiv, accessed May 17, 2026, <https://arxiv.org/html/2603.07496v1>
 39. Emergent Swarm Intelligence for Cybersecurity: A Hierarchical Autonomous Framework for Proactive Threat Detection and Incident Response - ResearchGate, accessed May 17, 2026, https://www.researchgate.net/publication/403943737_Emergent_Swarm_Intelligence_for_Cybersecurity_A_Hierarchical_Autonomous_Framework_for_Proactive_Threat_Detection_and_Incident_Response
 40. [2502.01822] Firewalls to Secure Dynamic LLM Agentic Networks - arXiv, accessed May 17, 2026, <https://arxiv.org/abs/2502.01822>
 41. Firewalls to Secure Dynamic LLM Agentic Networks - OpenReview, accessed May 17, 2026, <https://openreview.net/forum?id=w02FW1dMoY>
 42. Weekly TMLR digest for Mar 22, 2026 - Google Groups, accessed May 17, 2026, <https://groups.google.com/g/tmlr-announce-weekly/c/RXahm1n1pzi>
 43. Firewalls to Secure Dynamic LLM Agentic Networks - GitHub, accessed May 17, 2026, <https://github.com/amrgomaaelhady/Firewall-Agentic-Networks>
 44. Firewalls to Secure Dynamic LLM Agentic Networks - arXiv, accessed May 17, 2026, <https://arxiv.org/html/2502.01822v6>